Lab 01 — Install Kubernetes on Ubuntu with kubeadm

ID: K8S-LAB-01 · **Category:** Beginner · **Level:** Beginner · **Duration:** 90 min

Focus: kubeadm, kubelet, kubectl, containerd, Flannel, control plane, worker node

Objective

Stand up a single-node Kubernetes cluster on Ubuntu using kubeadm, containerd and Flannel, then deploy a Pod that proves the cluster works end-to-end.

Concepts Covered

• Why swap must be disabled for the kubelet

• Kernel modules and sysctl required by CNI

• Container runtime (containerd) vs Docker Engine

• What kubeadm, kubelet and kubectl each actually do

• PodCIDR and overlay networking (Flannel)

Prerequisites

• Ubuntu 22.04 LTS with sudo

• 2 vCPU, 2 GB RAM, 20 GB disk minimum

• Outbound internet to registry.k8s.io and ghcr.io

Architecture

kubeadm bootstraps the static pods of the control plane on the host. The kubelet on every node speaks to containerd (the CRI), which uses runc to start Linux containers. Flannel runs as a DaemonSet and programs an overlay so every Pod gets a routable IP from 10.244.0.0/16.

Lab Steps

01. Update Ubuntu and disable swap

Kubelet refuses to start when swap is on because the scheduler's memory accounting becomes incorrect once pages can be paged out.

```
sudo apt-get update && sudo apt-get -y upgrade
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

Expected: swap entries commented in /etc/fstab; `free -h` shows Swap: 0B

Internals: Cgroups v1/v2 memory limits assume RSS == working set. With swap enabled, OOMKill decisions become non-deterministic.

02. Load kernel modules and sysctl for CNI

br_netfilter lets iptables see bridged traffic so Service/Pod NAT rules apply. ip_forward enables Linux to route between Pod networks.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

Expected: `lsmod | grep br_netfilter` shows the module loaded.

Internals: kube-proxy programs iptables/IPVS rules on the host. Without bridge-nf-call-iptables those rules are skipped for bridged pod traffic and Services break silently.

03. Install containerd and configure systemd cgroup driver

Kubelet and containerd MUST agree on the cgroup driver. Kubernetes ≥1.22 defaults to systemd; mismatches cause subtle pod restarts.

```
sudo apt-get -y install containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
```

Expected: `sudo systemctl status containerd` → active (running).

Internals: containerd exposes the CRI socket at /run/containerd/containerd.sock. kubelet calls CRI RPCs (RunPodSandbox, CreateContainer) which containerd implements via runc.

04. Install kubeadm, kubelet and kubectl

kubeadm bootstraps the cluster, kubelet is the node agent, kubectl is the CLI. Pinning prevents an `apt upgrade` from breaking the control plane.

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/de
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Expected: `kubeadm version` and `kubectl version --client` print v1.30.x.

Internals: kubelet runs as a systemd unit (/etc/systemd/system/kubelet.service). It will crash-loop until kubeadm writes /var/lib/kubelet/config.yaml during init.

05. Initialize the control plane

kubeadm creates the CA, writes static pod manifests for api-server/etcd/controller-mgr/scheduler, and uploads cluster config.

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --cri-socket=unix:///run/containerd/containerd.sock
```

Expected: Ends with `Your Kubernetes control-plane has initialized successfully!` and a `kubeadm join` command. SAVE IT.

Internals: Static pods come from /etc/kubernetes/manifests/. Kubelet watches that directory and starts the pods directly — they're not scheduled.

06. Configure kubectl access

admin.conf holds the client cert for the cluster-admin user. Copy it to ~/.kube/config so kubectl can authenticate.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

Expected: Node shows `NotReady` — expected until a CNI is installed.

Internals: kubectl reads context, cluster and user blocks from ~/.kube/config and signs each request with the client cert listed in `users[].user.client-certificate-data`.

07. Install Flannel CNI

Flannel programs an overlay (VXLAN) that allocates a /24 of the PodCIDR to every node and routes Pod-to-Pod traffic across nodes.

```
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml
```

Expected: `kubectl -n kube-flannel get pods` shows kube-flannel-ds Running on the node.

Internals: Flannel writes /run/flannel/subnet.env and a CNI conflist under /etc/cni/net.d. Without that conflist, kubelet's PLEG can't set Pods to Ready.

08. Verify node and CoreDNS

Node should turn Ready within ~60s. CoreDNS pods leave ContainerCreating once the CNI is healthy.

```
kubectl get nodes
kubectl -n kube-system get pods
```

Expected: Node STATUS=Ready, coredns-* pods Running 1/1.

Internals: CoreDNS stays in ContainerCreating until kubelet can attach it to the Pod network. That requires a working CNI binary AND a populated /etc/cni/net.d/.

09. Allow workloads on the control plane (single-node lab)

By default the control plane is tainted NoSchedule. For a single-node lab we untaint it.

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

Expected: `taint "node-role.kubernetes.io/control-plane" removed`

Internals: Taints repel pods unless they carry a matching toleration. Removing the taint lets the scheduler place workload pods on this node.

10. Deploy a first Pod and prove it works

Runs an nginx Pod, shows its assigned PodIP, then proves the container actually serves traffic.

```
kubectl run web --image=nginx --port=80
kubectl get pod web -o wide
kubectl exec web -- curl -s localhost | head -5
```

Expected: Pod Running 1/1, IP from 10.244.0.0/16, curl returns `<!DOCTYPE html>`.

Internals: kubelet pulled the image via containerd → CRI ImagePull, created a sandbox (pause container) for the network namespace, then the nginx container joined it.

Common Errors

Node stuck NotReady — Cause: CNI not installed or /etc/cni/net.d empty · Fix: Apply Flannel/Calico, then `journalctl -u kubelet | tail`

CoreDNS ContainerCreating — Cause: CNI not ready yet · Fix: Wait for kube-flannel-ds Running, then `kubectl -n kube-system rollout restart deploy coredns`

Flannel CrashLoopBackOff — Cause: PodCIDR mismatch with `--pod-network-cidr` · Fix: Re-init cluster with the correct CIDR, or patch flannel ConfigMap

x509 certificate signed by unknown authority — Cause: Stale ~/.kube/config from a previous cluster · Fix: Re-copy /etc/kubernetes/admin.conf and `chown` it

Pending Pods on a fresh control plane — Cause: Control-plane taint still present · Fix: Untaint or add `tolerations` to the Pod spec

Troubleshooting

• Investigate the kubelet first: `sudo journalctl -u kubelet -f`

• Inspect runtime: `sudo crictl ps`, `sudo crictl logs <id>`

• Look at CNI: `ls /etc/cni/net.d/`, `cat /run/flannel/subnet.env`

• Validate API health: `kubectl get --raw=/readyz?verbose`

Enterprise Relevance

kubeadm is still the reference bootstrap path for on-prem and air-gapped enterprise clusters. Understanding the swap/cgroup/CNI triad makes you the engineer who recovers a stuck cluster at 2am instead of the one who pages someone else.

Validation

• `kubectl get nodes` shows Ready

• `kubectl get pods -A` shows no CrashLoopBackOff

• An nginx Pod gets a 10.244.x.x IP and responds to curl

Related Incidents & Guide Sections

Incidents: K8S-INC-3002

• Control plane & worker components

• Pod lifecycle & probes

Questions

Beginner

Q. Why must swap be disabled before kubelet starts?

A. Kubelet's memory accounting assumes pages are never paged out; swap makes OOMKill decisions non-deterministic.

Q. What is the difference between kubeadm and kubelet?

A. kubeadm is a one-shot bootstrapper that creates the cluster. kubelet is the long-running node agent that talks to the API server and the runtime.

Intermediate

Q. Why does `--pod-network-cidr=10.244.0.0/16` matter for Flannel?

A. Flannel's default config expects that exact CIDR. A mismatch causes Flannel CrashLoopBackOff and Pods that never get an IP.

Advanced

Q. Where do control-plane components actually run after `kubeadm init`?

A. As static pods, manifests in /etc/kubernetes/manifests/, started directly by kubelet without going through the scheduler.

Scenario

Q. After `kubeadm init` your node is Ready but CoreDNS is stuck ContainerCreating. Where do you look?

A. /etc/cni/net.d for an empty directory, then the CNI pod's logs. CoreDNS can't get a Pod IP without a working CNI.

Viva

Q. Walk me from `kubectl run web --image=nginx` to a running container.

A. kubectl → API server → etcd → scheduler binds Pod → kubelet on node → CRI RunPodSandbox → CNI assigns IP → CRI CreateContainer → containerd → runc → Linux namespaces/cgroups.